Software Requirements Specifications
Project Title: Micro-Service Hub

1. Introduction

This SRS describes what MicroServiceHub will do and how it is expected to perform. It follows the outline of the course: Introduction, Overall Description and Requirements. The functional requirements are written as user stories, each with a confirmation part (acceptance criteria) that tells how the story is checked.

1.1 Purpose

The purpose of **MicroServiceHub** is to give people in Bangladesh one trusted place to hire and offer everyday services, to pay safely, and to reach emergency help. Today, people find workers through phone calls and social media groups: they cannot check who they are dealing with, prices are unclear, payment is risky, and ambulances, doctors and blood donors must be searched for in different places.

The product is a web application (React front end, Express API, MySQL database) that is live at https://microservicehub.onrender.com. This document is the basis for design, implementation, testing and acceptance of version 1.0.

1.2 Intended Audience

The SRS is meant for the following readers.

Reader	How they use the SRS
Developers	Build exactly the behaviour in the user stories and follow the non-functional requirements.
Testers (quality assurance)	Turn each confirmation into a test case. The automated test suite already follows many of them.
Project manager and course teacher	Check that the scope is clear and complete, and judge the result against it.
Platform owner and admins (operations)	Check that daily work such as KYC review, disputes, payouts and emergency services is supported.
Maintainers	Understand why a rule exists before changing it.

1.3 Intended Use

This document should be used in the following ways:

- **Before development:** to agree on what is in version 1.0 and what is not.
- **During development:** as the checklist for each feature. A story is "done" only when all its confirmation points are met.
- **During testing:** as the source of test cases and the definition of expected results.
- **During maintenance:** to decide whether a change is a bug, a new requirement, or a feature to retire.

Each requirement has an identifier (US-n for user stories, NFR-n for non-functional requirements) and a priority: **High** (the product is not usable without it), **Medium** (important, but a workaround exists) or **Low** (a useful improvement).

1.4 Product Scope

MicroServiceHub lets **seekers** (people who need a service) and **providers** (people who offer one) find each other, agree on a job and a price, and pay with protection. Every user is verified with a National ID or birth certificate and a selfie. The platform also runs **emergency services** managed by the admin team: ambulances with live tracking, doctor checkups with an approved meeting link, and a blood bank.

Objectives and goals:

- Two ways to hire: post a work request and compare applications and offered prices, or browse listings and book a date.
- Trust: identity verification of all users reviewed by an admin, and reviews of providers.
- Safe payment in BDT: cash, bKash and Nagad confirmed by the provider, and online bKash held in escrow until the job is confirmed, with disputes decided by an admin.
- Communication: chat, voice calls and notifications (in-app, push, email, SMS).
- Location: an opt-in "Available now" map and consent-based live location sharing.
- Emergency help in one place, with the admin team in control.
- A complete admin panel for users, content, payments, fees, disputes and reports.

Benefits. Seekers get trusted help with clear prices and protected money. Providers get jobs, a reputation and reliable payment. The owners get a platform they can moderate and run from one panel.

In scope (version 1.0): accounts and identity verification; work requests, applications and hiring; listings and bookings; payments, escrow and disputes; chat, calls, notifications and location; emergency services; the admin panel.

Out of scope (version 1.0): native Android and iOS apps, a Bangla interface, automatic payouts to providers, automatic NID database checks, and video meetings inside the app (an external meeting link is used).

1.5 Risk Definitions

A risk is something that could go wrong and cause harm to users or to the platform. Defining the risks first makes it easier to see which requirements are needed to reduce them. The requirement numbers in the last column are the ones that reduce each risk (US = user story, NFR = non-functional requirement).

Table 1.1: Risk definitions

Risk	What could happen	How the SRS reduces it
R-1 Payment loss or fraud	A seeker pays and the provider does not work, or a provider works and is not paid. A forged payment callback marks an unpaid job as paid.	Escrow and disputes (US-20 to US-26, NFR-9), server-side check with the gateway (NFR-14)
R-2 Fake identity	A person with a false identity posts work, applies or books, and harms others.	KYC for every user and admin review (US-5 to US-7)
R-3 Privacy leak	Phone numbers, exact location or identity photos are seen by people who should not see them.	Consent rules and privacy limits (US-30, US-31, US-34, US-38, US-39, NFR-11, NFR-15, NFR-16)
R-4 Emergency help fails	A user waits for an ambulance that is not free, or is told wrong or old information, or treats the platform as a replacement for 999. A doctor checkup is used for a life-threatening condition.	Busy-state locking, live position, 999 warnings, admin-approved doctors (US-32 to US-40, NFR-6 to NFR-8)
R-5 Wrong medical or blood information	A fake doctor or a blood request that is old or false misleads people.	Admin adds doctors with BMDC number, requests expire in three days (US-35, US-37, US-39, NFR-7, NFR-10)
R-6 Abuse and spam	Fake or repeated posts, requests or messages flood the system.	Rate limits, limits on open requests and pending checkups, reports and moderation (US-11, US-36, US-39, US-42, US-43, NFR-17)
R-7 Account takeover	Someone guesses or steals a password or token.	Hashed passwords, token rules, reset by emailed link (US-2, US-3, NFR-12)
R-8 Data loss or downtime	The database or server fails and data or service is lost.	Managed database, automatic schema, backups, tests (NFR-23 to NFR-27)
R-9 Admin misuse	An admin removes or changes things they should not, or a sub admin harms the main admin.	Main admin protections and role checks (US-42, US-45, NFR-13)

2. Overall Description

Micro Service Hub is a **new, stand-alone product** (not an update or add-on to an earlier system). It is built for people in Bangladesh who need everyday services or offer them, and it adds emergency help because those needs often come together. It is a web application, so nothing has to be installed, and it can also be added to a phone’s home screen.

2.1 User Classes and Characteristics

Table 2.1: User classes

User class	Type	Characteristics
Seeker	Primary	Any adult or family who needs a job done. Many use a phone only. Technical skill from low to medium. Uses the system when a need comes up, and in emergencies.
Provider	Primary	A worker, professional or small business. Often has a phone and limited time. Needs quick ways to see new jobs and to be paid. May have a low level of English.
Admin	Primary (operations)	A trained team member. Uses a laptop or phone daily. Verifies identity, handles reports and disputes, and runs ambulances, doctors and the blood bank. Needs speed and clear information.
Main Admin	Primary (owner)	The owner of the platform. Has all admin powers plus management of admins, categories, fees, promo codes and payouts.
Guest	Secondary	A visitor who has not signed in. Reads public pages, contacts the team and registers.
Ambulance driver	Secondary	Does not have an account. Opens a secret link on a phone to share the ambulance position.
Doctor	Secondary (not a login user)	Is added and managed by admins, who talk to the doctor personally. The patient meets the doctor through the approved meeting link.
Platform owner (buyer)	Stakeholder	Decides fees, categories and policies and wants the platform to be safe, legal and sustainable.

2.2 User Needs

Table 2.2: Needs of each user class

User class	Main needs
Seeker	Find a trustworthy provider quickly; compare prices and reviews; pay without fear of loss; follow the job; get an ambulance, a doctor or blood fast, and be able to call 999 first.
Provider	Find jobs near them; apply with their own price; show their work and ratings; be sure that the money is real before starting; get a clear record of earnings and payouts.
Admin	See what needs attention at once (KYC, payments, disputes, reports); act in few steps; add, edit and remove ambulances, doctors and blood-bank entries; track ambulances live.
Main Admin	All of the above, and control over who is an admin, which service categories exist, the platform fee and promo codes, and payouts.
Guest	Understand what the platform does and how to join; contact the team.
Ambulance driver	Share the position with one tap, without creating an account.

Platform owner	Reduce fraud, keep users' data private, and have records for disputes and reports.

2.3 Operating Environment

Part	Environment
Client devices	Smartphones, tablets and computers with a current version of Chrome, Edge, Firefox or Safari; JavaScript enabled; internet connection. A camera is needed for identity photos, and location services for "Available now", ambulance tracking and live location (optional).
Application server	Node.js 22 with Express 5 on a Linux container (Render web service). It serves both the API and the built React files.
Database	MySQL-compatible: TiDB Cloud in production, MariaDB 10.4 on the development computer.
Real-time	Socket.IO over WebSocket, on the same server.
External services	bKash (payments), OpenStreetMap (map tiles), Web Push, SMTP email and an SMS gateway.
Development	Windows 11 laptop, VS Code, Git and GitHub; the live site is deployed automatically from the main branch.

2.4 Constraints

- **Time and team:** the product is built as a semester project by a small team, so features are limited to what can be built, tested and deployed in that time.
- **Free hosting:** the free Render service sleeps when idle (the first request can take up to a minute) and has limited memory; the free TiDB tier has storage and request limits.
- **Payment gateway:** a real bKash merchant account is needed for live online payments. Until it is available, the gateway is tested with a simulated one and users pay manually.
- **Messages:** email and SMS delivery need accounts at a mail or SMS provider.
- **Technology:** the front end is React with Vite, the back end is Node.js with Express, the data is in a MySQL-compatible database. Photos and identity documents are stored in the database.
- **Browsers and devices:** location, notifications and camera need the user's permission. A driver's phone must keep the tracking page open to share the position.
- **Language and currency:** the interface is in English and amounts are in BDT.
- **Law and policy:** the terms and privacy pages must be reviewed by a lawyer before a commercial launch. Emergency features support, but never replace, the national emergency number 999.
- **Security:** identity documents and phone numbers are sensitive; they must follow the privacy rules in this SRS.

2.5 Assumptions

The requirements in this SRS depend on the following assumptions. If one of them turns out to be false, the related requirements must be reviewed.

1. Users have a phone or computer, an internet connection and an email address, and a mobile number that belongs to them.
2. Admins review identity documents, disputes and checkup requests in reasonable time.
3. Providers and seekers give true identity documents; the admin can judge them by looking at the photos.
4. A bKash merchant account, an email provider and an SMS provider will be connected before real use.
5. Ambulance drivers agree to share the position from their phone while a trip is active, and the phone has GPS and internet.
6. Doctors are checked personally by the admin team, including the BMDC registration number, before they are added.
7. OpenStreetMap map tiles and the browser push services stay available free of charge.
8. The national emergency number 999 remains the first choice in a life-threatening emergency, and users are told so.
9. The database provider keeps regular backups of the data.

3. Functional Requirements

The functional requirements define the functions and services that the **Micro-Service Hub Platform** shall provide to its users and administrators. The system shall support user registration, NID verification, service posting, service discovery, service application, price negotiation, real-time communication, online payment, service completion, reviews, dispute management, and administration.

3.1 User Registration and Login

FR-01: The system shall allow users to create an account by providing required information such as name, email/phone number, and password.

FR-02: The system shall allow registered users to log in using their valid credentials.

FR-03: The system shall validate user credentials before granting access to the system.

FR-04: The system shall provide password reset functionality for users who forget their passwords.

FR-05: The system shall allow users to update their profile information.

3.2 User Role Management

FR-06: The system shall support two primary user roles:

- Service Seeker
- Service Provider

FR-07: A registered user shall be able to use the platform as a service seeker or service provider.

FR-08: The system shall provide appropriate functions and permissions according to the user's role.

3.3 NID Verification

FR-09: The system shall allow users to submit the required NID information for identity verification.

FR-10: The system shall verify the submitted identity information through an available KYC/NID verification service or sandbox API.

FR-11: The system shall display the user's verification status as **Verified, Pending, or Rejected**.

FR-12: The system shall allow users to view their own verification status.

FR-13: The system shall use the verification status to establish trust between service seekers and service providers.

3.4 Service Posting

FR-14: The system shall allow service seekers to create and post micro-service requests.

FR-15: A service post shall contain information such as:

- Service title
- Service description
- Service category
- Location
- Required date/time
- Offered amount
- Additional instructions

FR-16: The system shall allow service seekers to edit their service posts before a provider is selected.

FR-17: The system shall allow service seekers to cancel their service posts when permitted by the service status.

FR-18: The system shall display active service posts to eligible service providers.

3.5 Service Discovery and Search

FR-19: The system shall allow service providers to browse available micro-services.

FR-20: The system shall allow users to search for services using keywords and categories.

FR-21: The system shall provide location-based service discovery to help users find nearby services.

FR-22: The system shall allow users to filter services based on criteria such as category, location, budget, and service status.

FR-23: The system shall display relevant service information before a user applies for a service.

3.6 Service Application

FR-24: The system shall allow service providers to apply for available service requests.

FR-25: A service provider shall be able to submit an application containing a message and relevant information.

FR-26: The service seeker shall be able to view applications received from service providers.

FR-27: The service seeker shall be able to select a suitable service provider from the received applications.

FR-28: The system shall notify the selected service provider when their application is accepted.

FR-29: The system shall update the service status after a provider is selected.

3.7 Price Offer and Negotiation

FR-30: The service seeker shall specify an initial amount they are willing to pay for the requested service.

FR-31: The service provider shall be able to view the amount offered by the service seeker.

FR-32: The service provider shall be able to accept the offered amount.

FR-33: The service provider shall be able to submit a counter offer with a different amount instead of accepting the initial offer.

FR-34: The service seeker shall be able to accept or reject the counter offer.

FR-35: The service seeker shall also be able to submit another amount in response to a counter offer.

FR-36: The system shall allow both parties to continue price negotiation until an amount is mutually agreed upon.

FR-37: The system shall maintain the history of offers and counter offers exchanged between the two parties.

FR-38: The system shall determine the final service amount only after both parties agree on an amount.

FR-39: The system shall notify users when an offer or counter offer is submitted, accepted, or rejected.

3.8 Real-Time Chat and Communication

FR-40: The system shall provide a real-time chat facility between the service seeker and service provider.

FR-41: The service seeker and service provider shall be able to exchange text messages in real time.

FR-42: Users shall be able to discuss service requirements, price, timing, location, and other relevant details through the chat system.

FR-43: The system shall maintain the chat history associated with a service.

FR-44: The system shall restrict service-related chat access to the relevant service seeker and service provider.

FR-45: The system shall notify users when a new chat message is received.

3.9 Location and Map Services

FR-46: The system shall allow users to provide a location related to a service.

FR-47: The system shall display service locations using an integrated map service such as Google Maps or OpenStreetMap.

FR-48: The system shall support location-based matching between service seekers and nearby service providers.

FR-49: The system shall allow users to view relevant location information before applying for or accepting a service.

3.10 Payment Processing

FR-50: The system shall allow the service seeker to make an online payment after the final service amount has been agreed upon.

FR-51: The system shall integrate with a supported payment gateway for processing online payments.

FR-52: The system shall hold the agreed payment amount securely until the service is completed and confirmed.

FR-53: The system shall release the held payment to the service provider after successful confirmation of the completed service.

FR-54: The system shall record payment status and transaction information.

FR-55: The system shall notify users about successful, failed, pending, or released payments.

FR-56: The system shall support sandbox/test payment APIs for academic development.

3.11 Service Completion and Confirmation

FR-57: The service provider shall be able to mark an assigned service as completed.

FR-58: The service seeker shall be able to view the service completion status.

FR-59: The service seeker shall be able to confirm whether the service has been completed successfully.

FR-60: The system shall release the held payment after successful confirmation.

FR-61: The system shall update the service status to **Completed** after confirmation.

FR-62: The system shall prevent the payment from being released before the required completion confirmation.

3.1 Review System

FR-63: The system shall allow the service seeker to write a review for the service provider after completion of a service.

FR-64: The system shall allow the service provider to write a review for the service seeker after completion of a service.

FR-65: The system shall store reviews associated with completed services.

FR-66: The system shall display relevant reviews on user profiles.

FR-67: The system shall not use a numerical or star-based rating system.

3.2 Dispute and Report Management

FR-68: The system shall allow users to report inappropriate behavior, fraudulent activity, or service-related problems.

FR-69: The system shall allow users to raise a dispute regarding a service or payment.

FR-70: The system shall allow users to provide relevant details when submitting a dispute or report.

FR-71: The system shall store submitted reports and dispute information.

FR-72: The administrator shall be able to review and manage disputes and reports.

FR-73: The administrator shall be able to take appropriate action regarding disputes according to system policies.

3.3 Notifications

FR-74: The system shall provide notifications for important service-related events.

FR-75: The system shall generate notifications for events such as:

- New service application
- Application acceptance
- New price offer
- Counter offer
- Offer acceptance or rejection
- New chat message
- Payment status update
- Service completion request
- Service confirmation
- New review
- Dispute/report status update

3.4 Admin Dashboard

FR-76: The system shall provide a secure dashboard for administrators.

FR-77: The administrator shall be able to view and manage registered users.

FR-78: The administrator shall be able to monitor service posts and service activities.

FR-79: The administrator shall be able to monitor user verification statuses.

FR-80: The administrator shall be able to monitor payment and transaction information.

FR-81: The administrator shall be able to review reports, disputes, and complaints.

FR-82: The administrator shall be able to suspend or restrict accounts that violate system rules.

3.5 Service Status Management

FR-83: The system shall maintain the status of each service throughout its lifecycle.

FR-84: The service status shall include states such as:

- Posted
- Application Received
- Provider Selected
- Offer Submitted
- Counter Offer Submitted
- Price Agreed
- Payment Pending
- Payment Held
- In Progress
- Completed
- Confirmed
- Cancelled
- Disputed

FR-85: The system shall automatically update the service status based on relevant user actions.

3.6 Service History

FR-86: The system shall maintain a history of services posted, applied for, accepted, and completed by each user.

FR-87: Users shall be able to view their previous service activities.

FR-88: Users shall be able to view relevant offers, counter-offer, chat, payment, and review history associated with their services.

3.7 Overall System Workflow

FR-89: The system shall support the complete service workflow:

Register → Login → NID Verification → Post/Find Service → Apply → Initial Amount Offer → Accept/Counter Offer → Real-Time Chat → Price Agreement → Payment → Service Completion → Confirmation → Payment Release → Review

FR-90: The system shall ensure that the final payment amount is determined only after the service seeker and service provider mutually agree on an amount.

FR-91: The system shall ensure that the payment is processed according to the mutually agreed final amount.

FR-92: The system shall maintain the necessary records of the complete service transaction from service posting to service completion and review.

4. Non-Functional Requirements

4.1 Security

- User passwords and personal information must be stored securely.
- Unauthorized access to accounts and sensitive data must be prevented.
- NID and payment information must remain confidential.
- Proper authentication and access control must be implemented.

4.2 Performance

- The system should respond quickly to user requests.
- Search, service posting, and application processes should have minimal delay.
- The system should support multiple users simultaneously without major performance issues.

4.3 Usability

- The interface should be simple, clear, and user-friendly.
- Users should easily post, search, and apply for services.
- The platform should be responsive on desktop, tablet, and mobile devices.

4.4 Reliability

- User, service, application, and payment data must be stored accurately.
- The system should minimize data loss and maintain consistent service status.
- Transactions and service records should be reliable.

4.5 Availability

- The platform should be accessible whenever required.
- System downtime should be minimized.
- The system should handle temporary service or API failures appropriately.

4.6 Scalability

- The system should support increasing numbers of users and services.
- New service categories and features should be added without major architectural changes.

4.7 Maintainability

- The system should use modular and well-organized code.
- Bugs should be easy to identify and fix.
- Future updates and feature additions should be easy to implement.

4.8 Compatibility

- The platform should work on major modern browsers such as Chrome, Firefox, Edge.
- It should support desktop, laptop, tablet, and mobile devices.

4.9 Data Integrity

- The system must maintain accurate and consistent data.
- Invalid and duplicate data should be prevented.
- Service, application, and payment records must remain synchronized.

4.10 Privacy

- Personal, NID, and communication information must be protected.
- Sensitive information should only be accessible to authorized users.

4.11 Extensibility

- The system should allow future features such as mobile apps, AI-based matching, digital wallets, and additional payment methods.

4.12 Legal and Ethical Requirements

- User data must be handled responsibly.
- The system should support reporting of fraud, harassment, and other inappropriate activities.
- User safety and privacy should be maintained.